

Actividad: Server with MySQL2 TS

Construye una API REST de productos con Node.js, Express, TypeScript y MySQL2. Usa como referencia la organización del servidor vista en clase, pero escribe tu propia implementación.

Objetivo

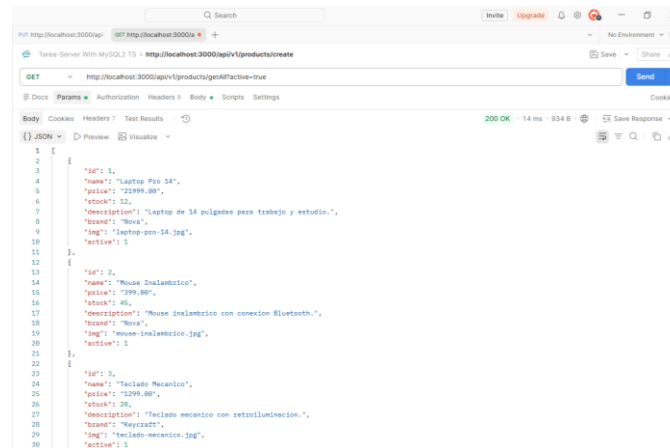
Crear un servidor que se conecte a una base de datos MySQL y permita consultar, crear, actualizar, dar de baja lógicamente productos y modificar sólo su precio. Las consultas normales no deben mostrar productos inactivos.

Evidencia de entrega

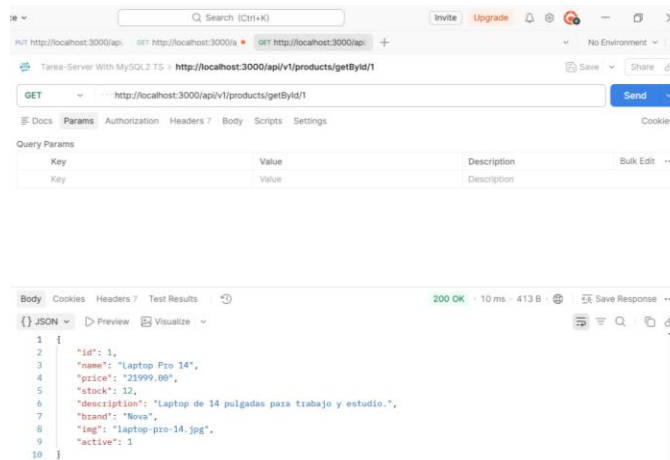
Entrega el enlace o archivo de una colección HTTP (Postman, Bruno, Thunder Client o equivalente), o capturas claras de las solicitudes y respuestas. Debe incluir evidencia de:

- consulta general y consulta por ID de un producto activo;

<http://localhost:3000/api/v1/products/getAll?active=true> (GET)

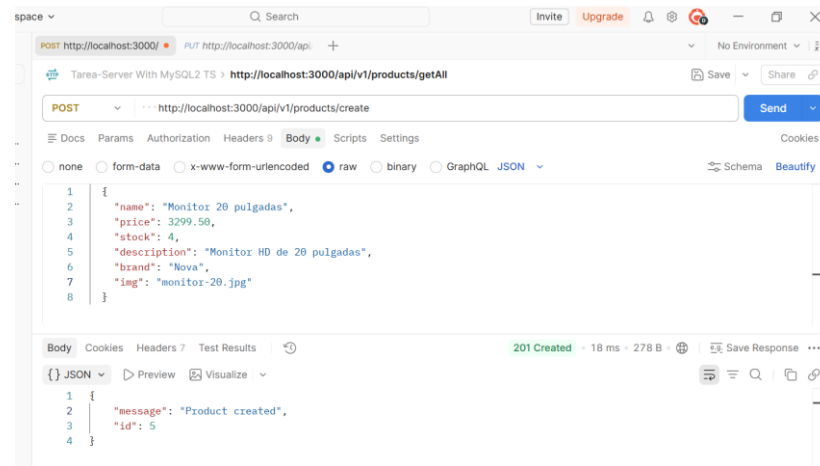


<http://localhost:3000/api/v1/products/getById/1> (GET)



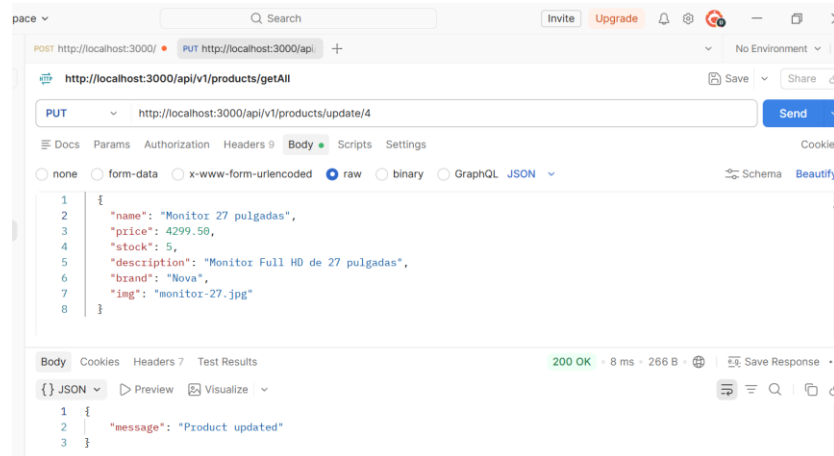
2. creación de un producto;

<http://localhost:3000/api/v1/products/create>

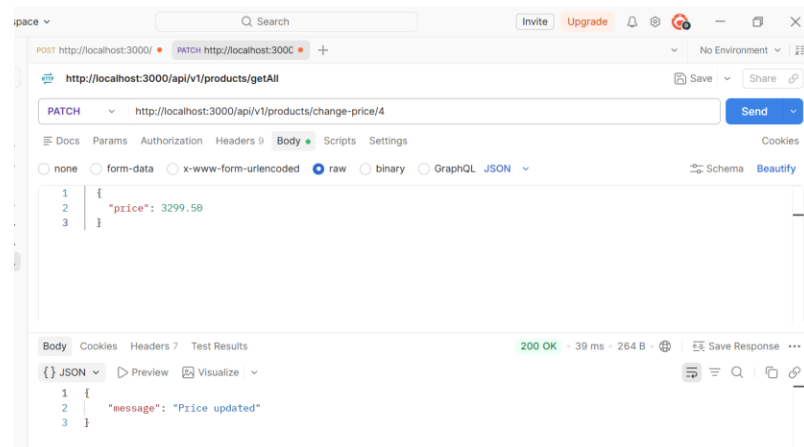


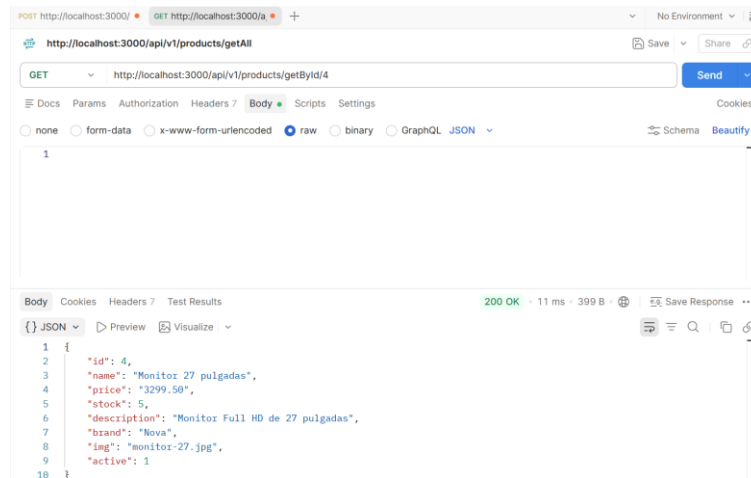
3. actualización completa y cambio de precio del producto creado;

<http://localhost:3000/api/v1/products/update/4>



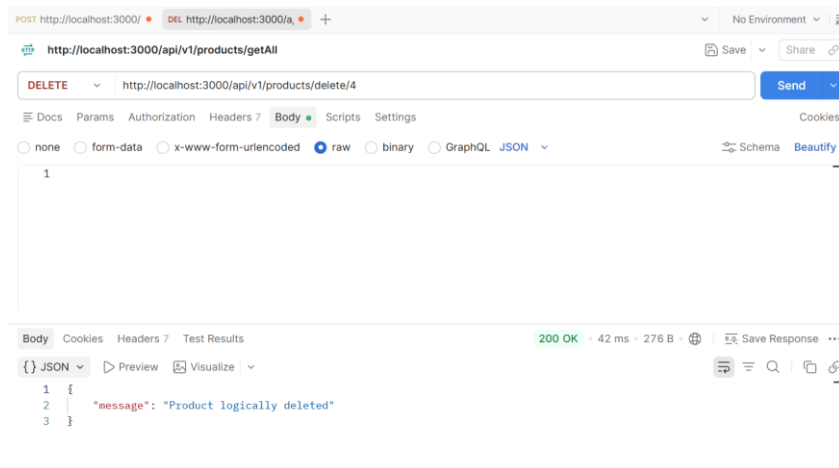
<http://localhost:3000/api/v1/products/change-price/4>





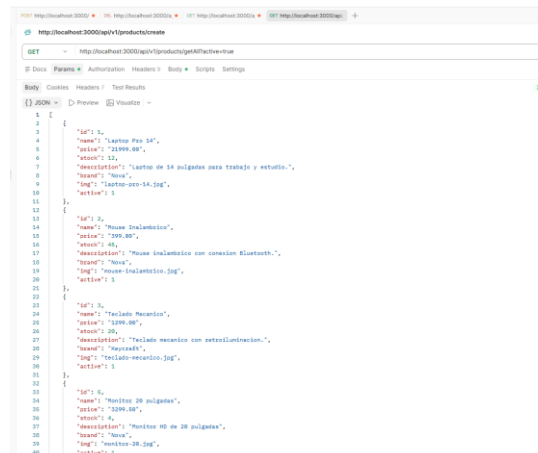
4. baja lógica del producto;

<http://localhost:3000/api/v1/products/delete/4>

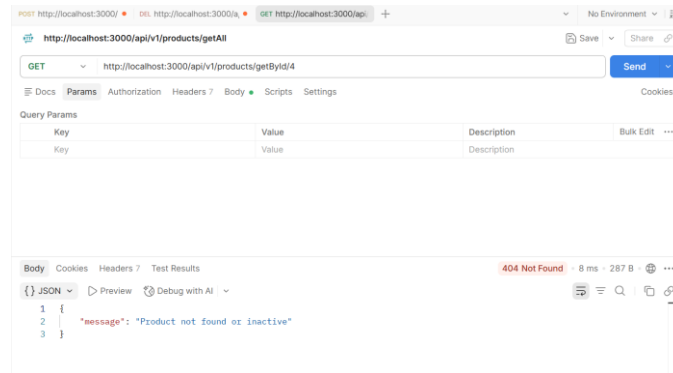


5. comprobación de que el producto dado de baja ya no aparece en getAll ni en getById:id;

Comprobacion de getAll: <http://localhost:3000/api/v1/products/getAll?active=true>

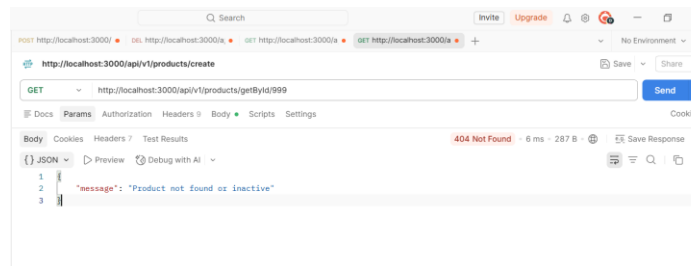


Comprobar baja By Id: <http://localhost:3000/api/v1/products/getById/4>

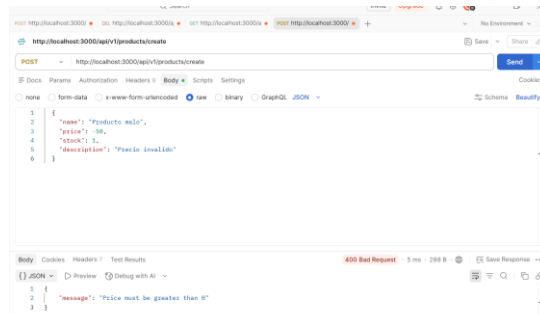


6. un caso de ID inexistente y un caso de datos inválidos.

Id inexistente: <http://localhost:3000/api/v1/products/getById/999>



Caso de datos inválidos: <http://localhost:3000/api/v1/products/create>



<http://localhost:3000/api/v1/products/change-price/1>

